WorkNest: Project Guide

Oct 12, 2026 · @Cobi

What WorkNest is

WorkNest is a job board and recruitment platform: candidates find and apply for jobs, and a recruitment team manages roles, client companies and applicants from an admin dashboard.

It is built as two separate projects that talk over HTTP:

- **Frontend** (`WorkNestFront`): a React website the user sees in the browser.
- **Backend** (`WorkNestBack`): an Express API that stores data in MongoDB, sends emails and handles file uploads.

Who uses it

User	What they do	Role in the database
Visitor	Browses and searches open jobs without an account	none
Candidate	Signs up, builds a profile, saves jobs, applies, tracks applications	`candidate`
Recruiter	Uses the admin dashboard to manage jobs and review applicants	`recruiter`
Admin	Everything a recruiter can do	`admin`

One-line pitch: WorkNest connects job seekers with companies, and gives recruiters one place to post roles, track every applicant through a hiring pipeline and report on placements.

Features

WorkNest has two sides: public and candidate pages, and an admin dashboard for the recruitment team.

Candidate side

Page	What it does
Home (`/`)	Landing page: hero search bar, how it works, jobs open this week, success stories, testimonials, FAQ
Sign up, Log in, Verify email, Forgot password	Account creation with a 6-digit email code, password reset, and "Continue with Google"
Find Jobs (`/find-jobs`)	Lists every open job; search by title, company or skill, filter by job type, work arrangement, experience and location, sort by closing date or salary
Job details (`/job-details/:id`)	Full job description, company info, save button, Apply button
Apply (`/apply/:id`)	A 4-step wizard: your details, CV and links, screening questions, review. Pre-fills from the profile
My Profile and Edit Profile	Headline, location, experience, skills, CV upload, job preferences, salary range, availability, with a completion score
Applications (`/applications`)	Tracks each application through its stages, lists saved jobs, and shows notifications such as "a saved job closes soon"

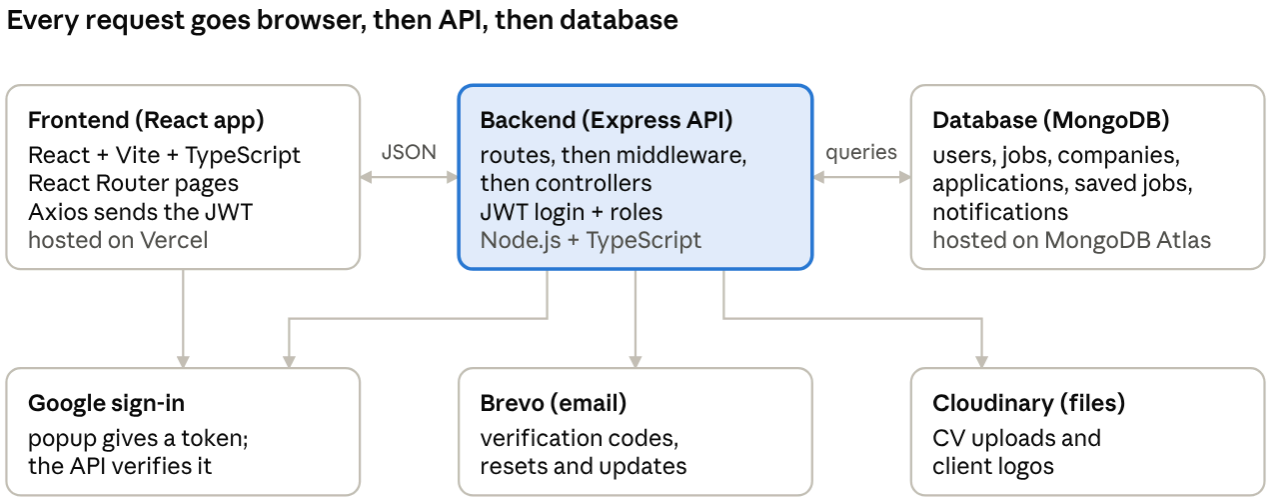
Admin side (admins and recruiters only)

Page	What it does
Overview (`/admin`)	Headline stats, recent applications, applications by status, jobs closing soon, most-applied roles, pipeline snapshot
Jobs	Every job in every status, with a New Role form that includes custom screening questions
Clients	The companies WorkNest hires for, with open roles, placements and a logo upload
Candidates	Every registered candidate, filterable by experience, availability and status (Open to work, Placed, Employed)

Page	What it does
Pipeline	Review each application: move it through stages, score it on a scorecard, leave internal notes

How it fits together

WorkNest is a classic three-tier web app: a React frontend, an Express REST API, and a MongoDB database.



WorkNest architecture · 3 core parts, 3 outside services

The browser never talks to the database directly; every read and write goes through the API, which checks who you are first.

What happens when a candidate opens Find Jobs

1. React Router shows the `FindJobs` page, which calls `fetchJobs()` in `src/api/jobs.ts`.
2. Axios sends `GET /api/jobs` to the server. If the user is logged in, it adds their token in an `Authorization: Bearer` header.
3. Express matches the route in `routes/jobRoutes.ts` and runs `getJobs` in `controllers/jobController.ts`.
4. The controller asks MongoDB, through the Mongoose `Job` model, for open jobs whose deadline hasn't passed.
5. The server sends the jobs back as JSON, and React draws a `JobCard` for each one.
6. Search and filters then run in the browser on that list, so results update as you type.

Frontend: libraries and what they do

The frontend is a single-page app written in **TypeScript** with **React 19**, built and served by **Vite**, and styled with **Tailwind CSS**.

Library	Version	What it does in WorkNest	Where to see it
react, react-dom	19.2	Builds the UI out of components (pages, cards, modals) and puts it on the page	`src/main.tsx`, every `.tsx` file
react-router-dom	7.18	Maps URLs to pages (`/find-jobs`, `/admin/candidates`) without reloading the page	`src/App.tsx`
axios	1.5	Sends HTTP requests to the API; one shared client adds the login token to every request	`src/lib/apiClient.ts`, `src/api/*.ts`
tailwindcss, @tailwindcss/vite	4.3	Utility classes for styling (`rounded-full bg-[#6D4AFF]`) written directly in the JSX	`src/index.css`, `vite.config.ts`
@react-oauth/google	0.13	The "Continue with Google" popup; returns a Google access token	`src/main.tsx`, `Login.tsx`, `SignUp.tsx`
react-toastify	11.1	The pop-up success and error messages (toasts)	`src/components/Toaster.tsx`, `src/lib/toast.tsx`
react-icons	5.7	Icon set (Feather icons such as `FiUser`, `FiBriefcase`)	`Navbar.tsx`, `AdminLayout.tsx`

React ideas used across the code

- **Components and props:** reusable pieces like `JobCard`, `Stepper` and `StatusPill`.
- **State with `useState`:** form inputs, filters, loading flags.
- **Side effects with `useEffect`:** loading data when a page opens.

- **Context:** `AuthContext` shares who is logged in with every page; `SavedJobsContext` shares which jobs are bookmarked.
- **Protected routes:** `ProtectedRoute` sends logged-out users to Log in, and non-admins away from `/admin`.
- **URL state:** Find Jobs keeps the search text in the URL (`?q=designer`), so a search can be shared.

Backend: libraries and what they do

The backend is a **Node.js** REST API written in **TypeScript** with **Express 5**, storing data in MongoDB through **Mongoose**.

Library	Version	What it does in WorkNest	Where to see it
express	5.2	The web server: receives requests, runs middleware, sends JSON responses	`src/server.ts`, `src/routes/*.ts`
mongoose	9.9	Defines the data models (User, Job…) and talks to MongoDB	`src/config/db.ts`, `src/models/*.ts`
jsonwebtoken	9.0	Creates the login token (JWT) and checks it on every protected request	`authController.ts`, `middleware/auth.ts`
bcryptjs	3.0	Hashes passwords and email codes so the real values are never stored	`models/User.ts`, `utils/otp.ts`
cors	2.8	Lets the frontend, on a different address, call the API	`src/server.ts`
dotenv	17.4	Loads secrets (database URL, API keys) from the `.env` file	`src/server.ts`
multer	2.3	Reads uploaded files (CVs, logos) from form submissions, in memory	`middleware/upload.ts`

Library	Version	What it does in WorkNest	Where to see it
cloudinary	2.11	Stores uploaded CVs and logos in the cloud and returns a link	`config/cloudinary.ts`
@getbrevo/brevo	6.0	Sends transactional emails through Brevo	`services/emailService.ts`

How a request moves through the server

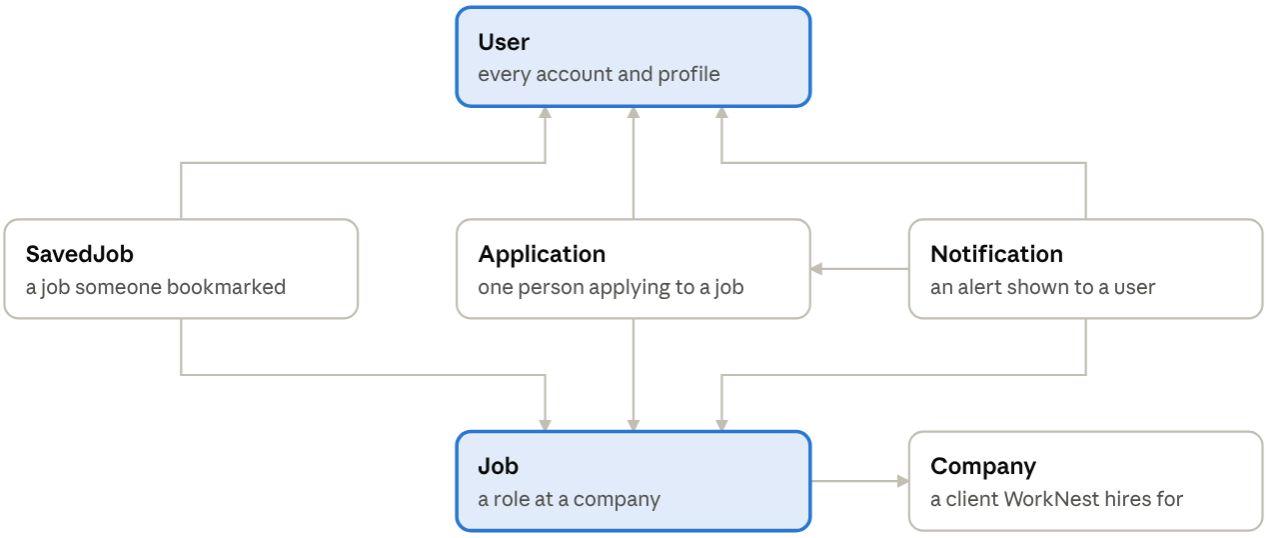
1. **Route** (`routes/jobRoutes.ts`): matches the URL and method, such as `GET /api/jobs` .
2. **Middleware** (`middleware/auth.ts`): `protect` checks the token, `authorize("admin", "recruiter")` checks the role, and `identify` reads a token if there is one.
3. **Controller** (`controllers/jobController.ts`): the actual logic. It queries the database and shapes the response.
4. **Model** (`models/Job.ts`): the Mongoose schema that validates and saves the data.

The server also has helper scripts run with `tsx` : `seed` (sample jobs and companies), `seed-candidates` (demo candidates), `make-admin` (promote an account), `fix-closing-dates` and `backfill-company-details` (one-off data fixes).

Database: the six models

MongoDB stores six kinds of record, each defined by a Mongoose model in `server/src/models` .

User and Job sit at the centre; everything else points at them



data model · 6 Mongoose models, arrows point to the record referenced

Each arrow is a reference: the record stores the other record's id, and Mongoose's `.populate()` swaps that id for the full record when the API needs it.

Model	Key fields	Rules worth mentioning
User	name, email, password, role, isVerified, headline, location, yearsOfExperience, skills, cvUrl, availability, openToWork	Email is unique; password is hashed before saving; code fields are hidden from every query
Company	name, slug, logoUrl, website, about, industry, location	Slug is unique, used in URLs
Job	company, title, description, skills, screeningQuestions, workArrangement, jobType, experienceLevel, salaryMin/Max, closesAt, status	Status is draft, open, closed or archived; currency defaults to ₦
Application	job, applicant, contact details, CV, whyThisRole, screeningAnswers, status, statusHistory, scorecard, internalNote	One account can apply to a job only once; guests can apply without an account
SavedJob	user, job	The pair is unique, so a job can't be saved twice

Model	Key fields	Rules worth mentioning
Notification	user, type, title, body, isRead, job, application	Types: application status change, saved job closing soon, new job match

Application stages: submitted, review, shortlisted, interview, offer, hired, or rejected. Every change is logged in `statusHistory`, which is how the candidate sees a timeline.

API endpoints

The API has about 40 endpoints under `/api`, plus a health check. Access levels: **Public** needs no login, **Login** needs a valid token, **Optional** works with or without one, **Staff** needs an admin or recruiter account.

Area	Method and path	Access	What it does
Health	GET `/api/health`	Public	Confirms the server is running
Jobs	GET `/api/jobs`	Public	Lists open jobs whose deadline hasn't passed
Jobs	GET `/api/jobs/:id`	Public	One job with its company
Auth	POST `/api/auth/register`	Public	Creates an account and emails a 6-digit code
Auth	POST `/api/auth/verify-email`	Public	Checks the code and logs the user in
Auth	POST `/api/auth/resend-otp`	Public	Sends a new code
Auth	POST `/api/auth/login`	Public	Email and password login
Auth	POST `/api/auth/google`	Public	Google login or sign-up
Auth	POST `/api/auth/forgot-password`, `/verify-reset-otp`, `/reset-password`	Public	The 3-step password reset

Area	Method and path	Access	What it does
Auth	GET `/api/auth/me`	Login	Returns the logged-in user
Profile	GET, PATCH `/api/profile/me`	Login	Read or update my profile
Profile	POST `/api/profile/me/cv`	Login	Uploads a CV to Cloudinary
Saved jobs	GET `/api/saved-jobs`, GET `/api/saved-jobs/ids`	Login	My saved jobs, or just their ids
Saved jobs	POST, DELETE `/api/saved-jobs/:jobId`	Login	Save or unsave a job
Applications	GET `/api/applications`	Login	My applications
Applications	GET `/api/applications/prefill`, `/job/:jobId`	Login	Pre-fill the apply form; check if I already applied
Applications	POST `/api/applications/:jobId`	Optional	Submit an application, as a guest or logged in
Notifications	GET `/api/notifications`, PATCH `/read-all`, PATCH `/:id/read`	Login	List notifications and mark them read
Admin: applications	GET `/api/admin/applications`, GET `/:id`	Staff	All applications, or one in detail
Admin: applications	PATCH `/api/admin/applications/:id/status`, `/scorecard`	Staff	Move a stage (emails the candidate) or score them
Admin: jobs	GET, POST `/api/admin/jobs`	Staff	Jobs in every status; create a job
Admin: clients	GET `/api/admin/companies`, `/clients`, `/clients/:id`	Staff	Company list, client cards, client detail
Admin: clients	POST, PATCH `/api/admin/companies/clients`, POST `/clients/logo`	Staff	Create or edit a client; upload its logo

Area	Method and path	Access	What it does
Admin: candidates	GET `/api/admin/candidates`	Staff	Every candidate with application count and status
Dashboard	GET `/api/dashboard/overview`	Staff	All the numbers on the admin Overview page

The API speaks JSON and follows REST conventions: GET reads, POST creates, PATCH updates part of a record, DELETE removes.

Authentication and security

WorkNest logs users in with a signed token (a JWT), and checks that token and the user's role on every protected request.

Signing up and logging in

1. The user signs up with name, email and password. The server hashes the password with **bcrypt** (cost 10) and never stores the real one.
2. The server emails a **6-digit code**. The code expires after **10 minutes**, allows **5 wrong attempts**, and is stored only as a hash.
3. Once the code is correct, the server creates a **JWT** containing the user's id and role, valid for **7 days**.
4. The browser keeps the token in `localStorage` (key `worknest_token`). Axios sends it on every request as `Authorization: Bearer <token>`.
5. When the app opens, it calls `GET /api/auth/me` to check the token is still valid and load the user.

Google sign-in: the Google popup returns an access token. The server checks it with Google (`oauth2/v3/tokeninfo`) and confirms it was issued for WorkNest's own Client ID before trusting it.

Password reset: request a code, enter it, then set a new password: three steps, each checked on the server.

Roles and protection

Layer	Where	What it stops
`protect` middleware	server, `middleware/auth.ts`	Requests with no token, or an expired or forged one
`authorize("admin", "recruiter")`	server	Candidates calling admin endpoints, even with a valid token
`ProtectedRoute`	frontend, `components/ProtectedRoute.tsx`	Logged-out users opening `/profile`; candidates opening `/admin`
Field whitelist	`profileController.ts`	A user changing their own role or email through the profile form
`select: false`	`models/User.ts`	Password and code hashes ever appearing in an API response

The frontend check is for user experience only; the real security is on the server, because anyone can call the API directly.

Secrets live in `.env` files that are git-ignored: the MongoDB URL, JWT secret, and the Brevo, Cloudinary and Google keys.

External services

WorkNest relies on five outside services, all with free tiers; only the database is required to run it.

Service	What WorkNest uses it for	Settings it needs	If it's missing
MongoDB Atlas	Cloud database holding all six collections	`MONGO_URI`	The server won't start

Service	What WorkNest uses it for	Settings it needs	If it's missing
Brevo	Emails: verification code, password reset, welcome, application received, status changes	`BREVO_API_KEY`, `BREVO_SENDER_EMAIL`, `BREVO_SENDER_NAME`	Codes print in the server terminal instead
Cloudinary	Stores CVs (PDF, DOC, DOCX up to 5 MB, folder `worknest/cvs`) and client logos (PNG, JPEG, WEBP, SVG up to 2 MB, folder `worknest/logos`)	`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Uploads fail; everything else works
Google Cloud (OAuth)	The "Continue with Google" button	`VITE_GOOGLE_CLIENT_ID` (frontend), `GOOGLE_CLIENT_ID` (server)	Only Google sign-in fails
Vercel	Hosts the frontend; `vercel.json` sends every URL to `index.html` so React Router can handle it	none in code	Run locally instead

Uploads never touch the server's disk: **multer** keeps the file in memory and streams it straight to Cloudinary, which returns a link that is saved on the user or company.

Developer tools

These tools are used to write, check and run the code, but are not part of what users download.

Tool	Used in	What it does
Node.js and npm	both	Runs JavaScript outside the browser; npm installs libraries and runs scripts
TypeScript	both	JavaScript with types, so mistakes like a missing field are caught before the code runs

Tool	Used in	What it does
Vite	frontend	Dev server with instant reload on `localhost:5173`, and the production build
ESLint (+ react-hooks, react-refresh plugins)	frontend	Flags bugs and bad patterns, such as a missing `useEffect` dependency
tsx	server	Runs TypeScript directly; `tsx watch` restarts the server on every save
Git and GitHub	both	Version control; two repos, `WorkNestFront` and `WorkNestBack`
VS Code	both	The code editor
Postman or a browser	server	Optional: call endpoints by hand while testing, e.g. open /api/health

npm scripts

Command	Project	What it runs
`npm run dev`	frontend	Vite dev server on port 5173
`npm run build`	frontend	Type-check with `tsc`, then a production build into `dist/`
`npm run lint`	frontend	ESLint over the project
`npm run dev`	server	`tsx watch src/server.ts` on port 5000
`npm run build`, `npm start`	server	Compile to `dist/`, then run `node dist/server.js`
`npm run seed`	server	Wipes and refills jobs and companies with sample data
`npm run seed-candidates`	server	Adds 14 demo candidates; `-- --remove` deletes them
`npm run make-admin <email>`	server	Gives an account the admin role

`nodemon` and `ts-node` are listed in the server's package.json but no longer used; `tsx` replaced them.

Folder structure

Both projects separate code by job: pages and components in the frontend, routes, controllers and models in the backend.

Frontend (`WorkNestFront`)

```
public/images/        logos, illustrations, photos
src/
  main.tsx            starts the app: Google, Router, Auth and SavedJobs
providers
  App.tsx             every URL and the page it shows
  api/                one file per area; each function makes one API call
  components/         reusable pieces: Navbar, JobCard, Stepper,
ProtectedRoute...
  context/            AuthContext (who is logged in), SavedJobsContext
  lib/                apiClient (axios + token), toast helpers
  Layout/             AuthLayout, shared by the login and sign-up pages
  pages/              one folder or file per page; Admin/ holds the dashboard
  types/              TypeScript shapes: Job, Application, Candidate...
  utils/              small helpers: relative dates, profile completion
  Data/               mock jobs used on the home page
vite.config.ts        Vite + Tailwind + React plugins
vercel.json           send every URL to index.html
```

Backend (`WorkNestBack`)

```
src/
  server.ts           creates the Express app, mounts every route, connects
to MongoDB
  config/             db.ts (MongoDB), cloudinary.ts
  routes/             URL to controller mapping, with middleware
  controllers/        the logic for each endpoint
  middleware/         auth.ts (protect, authorize, identify), upload.ts
(multer)
  models/             the six Mongoose models
  services/           emailService.ts (Brevo templates and sending)
  types/              shared enums, Express request typing
  utils/              otp.ts (create, hash and check codes)
  seed.ts, seedCandidates.ts, makeAdmin.ts   helper scripts
.env.example          every setting the server needs
```

Presenting WorkNest

A 10-minute talk fits in 8 slides and a 3-minute live demo.

Suggested slide outline

1. **The problem:** job seekers juggle scattered listings; recruiters track applicants in spreadsheets.
2. **The solution:** WorkNest, one place to find, apply and hire. Show the home page.
3. **Who uses it:** visitor, candidate, recruiter and admin (the table in "What WorkNest is").
4. **Architecture:** the diagram in "How it fits together".
5. **Tech stack:** React, TypeScript, Vite and Tailwind on the front; Express, Mongoose and MongoDB on the back.
6. **Data model:** the six-model diagram.
7. **Security:** hashed passwords, email codes, JWT, role checks on the server.
8. **What we learned and what's next.**

Demo script

1. Search Find Jobs for "designer" and filter by Remote.
2. Open a job, click Apply, and walk through the 4 steps.
3. Open Applications to show the new application and its stage.
4. Log in as an admin, open Pipeline, and move that application to Shortlisted.
5. Back on the candidate account, show the status change and the notification.
6. Finish on the admin Candidates page and the Overview dashboard.

Run `npm run seed` and `npm run seed-candidates` before presenting so every page has data, and keep both servers running.

Questions you may be asked

Question	Short answer
Why two separate projects?	The frontend and API can be deployed, scaled and changed independently; a mobile app could reuse the same API.
Why MongoDB instead of SQL?	Job and profile data varies in shape (skills lists, screening answers), which fits documents well; Mongoose still enforces a schema.
How do you stop a candidate opening admin pages?	The page is hidden by `ProtectedRoute`, but the real check is `authorize` on the server, which rejects their token.
Where are passwords stored?	Only as bcrypt hashes; the real password is never saved.

Question	Short answer
Is search done on the server?	No, the server sends all open jobs once and the browser filters them as you type. Fine for hundreds of jobs; thousands would move search to the server with pagination.
What happens if email isn't configured?	Sign-up still works: the code is printed in the server terminal.
What would you add next?	Server-side search and pagination, profile photos, and recruiter accounts scoped to their own clients.